

Expert Systems and Ontologies

From Rule-Based Reasoning to Modern Knowledge Graphs

Intelligent Systems Module

Where are we in the Intelligent Systems Module?

- **Past:** Search Algorithms (A^*), Optimization (LP/DP).
- **Last Lecture:** Formal Logic (Propositional & First-Order Logic), Resolution, Unification.
- **Today:** How do we package logic into usable software? We look at **Expert Systems** and **Ontologies**.

Learning Objectives

By the end of this lecture, you should be able to:

- ① **Understand** the historical shift from pure logic to rule-based Expert Systems.
- ② **Identify** the core architectural components of an Expert System.
- ③ **Explain** the Rete Algorithm and Conflict Resolution.
- ④ **Define** Ontological Engineering, RDF, and OWL.
- ⑤ **Evaluate** the Knowledge Acquisition Bottleneck.
- ⑥ **Analyze** modern alternatives: Knowledge Graphs and Retrieval-Augmented Generation (RAG).

Part 1: The Rise of Expert Systems

Moving from Abstract Logic to Domain Expertise

The Limits of Pure Logic (1950s–1960s)

The Era of General Problem Solvers

- Early AI focused on symbolic reasoning, formal logic, and general search algorithms.
- **The Goal:** Create a single algorithm that could solve *any* problem given the right logical axioms.
- **The Flaw:** Combinatorial explosion. Proving theorems in First-Order Logic is Semi-Decidable. They lacked the "common sense" to solve messy, real-world problems efficiently.

The Paradigm Shift (1970s)

Researchers realized that human experts (doctors, mechanics, chemists) rarely use pure mathematical logic.

- They use **Heuristics** (rules of thumb) and **Domain-Specific Knowledge**.
- *The Insight:* "In the knowledge lies the power." AI needed to stop trying to be generally smart, and start being highly specialized.
- **The Result:** The birth of the **Expert System**.

What is an Expert System?

Definition

An Expert System is an AI program that emulates the decision-making ability of a human expert by reasoning through bodies of knowledge, represented mainly as **IF-THEN rules**.

Why were they revolutionary?

- **Interpretable:** The system could explain *why* it made a decision.
- **Modular:** You could add a new rule without rewriting the whole program.
- **Accessible:** Domain experts could write rules without knowing C++ or Assembly.

Case Study 1: DENDRAL (1965)

The Pioneer of Expert Systems

- **Domain:** Organic Chemistry.
- **Task:** Identify unknown organic molecules by analyzing their mass spectrometry and nuclear magnetic resonance data.
- **Mechanism:** Used a heuristic rule base derived from expert chemists to prune the massive search space of possible molecular structures.
- **Result:** It was the first system to demonstrate that specialized domain knowledge beats generalized search. It outperformed human post-docs.

Case Study 2: MYCIN (1976)

The Medical Diagnostic System

- **Domain:** Medicine (Stanford University).
- **Task:** Diagnosed severe blood infections and recommended antibiotic dosages based on patient weight.
- **Mechanism:** Used over 600 rules. Introduced **Certainty Factors** (CF) to handle uncertainty (e.g., "IF symptom X, THEN disease Y is 0.8 likely").
- **Result:** Achieved an accuracy of 69%, higher than infectious disease experts. Was never deployed due to ethical and liability concerns.

The Commercial Success

- **Domain:** Computer Hardware Manufacturing.
- **Task:** Used by Digital Equipment Corporation (DEC) to automatically configure customer computer orders (checking for missing cables, incompatible parts).
- **Mechanism:** A massive Forward-Chaining system.
- **Result:** Handled 80,000 orders a year. Saved the company \$40 million annually. Sparked the corporate AI boom of the 1980s.

Part 2: Anatomy of an Expert System

Separation of Knowledge and Control

The Separation of Concerns

In traditional programming, data and control logic are tightly woven together.

```
if (patient.hasFever) { prescribeAspirin(); }
```

Expert systems introduced a strict **Separation**:

- 1 **Knowledge Base (The "What")**: Contains rules and facts. Entirely declarative.
- 2 **Inference Engine (The "How")**: The generic algorithm that applies the rules.

Benefit: Swap the Medical KB for a Financial KB, keep the same Inference Engine, and you instantly have a new AI!

The 5 Core Components

- 1 **Knowledge Base:** IF-THEN rules.
- 2 **Working Memory:** Database of current session facts.
- 3 **Inference Engine:** Matches facts against rules.
- 4 **Explanation Facility:** Explains "Why" and "How" to the user.
- 5 **User Interface:** Prompts the user for missing data.

Rule Structure

IF (Antecedents)

- condition 1
- condition 2

THEN (Consequent)

- action / new fact

The Knowledge Base vs. Working Memory

Knowledge Base (Long-term Memory)

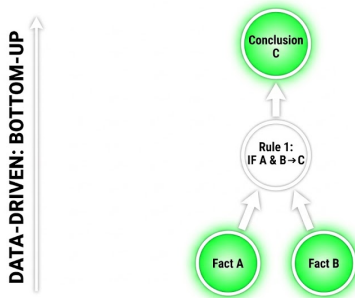
- Static, rarely changes during execution.
- Holds universal truths of the domain.
- Example: Rule 1: IF Engine=Hot AND Coolant=Low THEN Alert=Overheating

Working Memory (Short-term Memory)

- Highly dynamic, updates every millisecond.
- Holds the state of the current problem.
- Example: Fact 1: Engine=Hot. Fact 2: Coolant=Low.

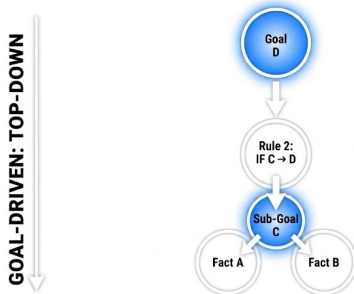
Inference: Forward Chaining (Data-Driven)

- **Direction:** Bottom-Up. (Facts $\rightarrow$ Rules $\rightarrow$ Conclusions).
- **Process:** The Engine scans Working Memory. It finds a rule where the 'IF' part is satisfied. It "fires" the rule, adding the 'THEN' part to Working Memory as a new fact.
- **Best for:** Planning, monitoring, configuration (e.g., XCON). You have all the data up front, and want to see what happens.



Inference: Backward Chaining (Goal-Driven)

- **Direction:** Top-Down. (Goal $\rightarrow$ Rules $\rightarrow$ Facts).
- **Process:** The Engine starts with a Hypothesis. It looks for rules that conclude that goal. It checks if the 'IF' conditions are true. If not, they become new sub-goals. If facts are missing, it asks the user.
- **Best for:** Diagnostics, troubleshooting (e.g., MYCIN). You have a goal, and want to find the evidence to support it.



The Rete Algorithm (Efficiency)

The Problem: If you have 10,000 rules and 1,000 facts, checking every rule against every fact continuously is $O(N \times M)$. The system would grind to a halt.

The Solution: The Rete Algorithm (Charles Forgy, 1979)

- Instead of checking rules against facts, it compiles the rules into a highly efficient **Tree Network**.
- It remembers partial matches. When a new fact arrives, it only updates the specific branches of the tree affected by that fact.
- Makes rule-based systems infinitely scalable. Used in modern business rule engines like JBoss Drools.

What happens if multiple rules are triggered at the exact same time? Which one fires first?

- ➊ **Refractoriness:** A rule should not fire twice on the exact same set of facts (prevents infinite loops).
- ➋ **Specificity:** Fire the most specific rule. (e.g., A rule with 5 conditions overrides a general rule with 1 condition).
- ➌ **Recency:** Fire the rule that uses the most recently added fact in Working Memory (keeps the AI focused on the current line of thought).
- ➍ **Priority / Salience:** The programmer manually assigns a weight (1-100) to rules.

Part 3: The AI Winter

Why did Expert Systems fall out of favor?

The Knowledge Acquisition Bottleneck

Extracting knowledge from a human expert and translating it into strict IF-THEN rules is the hardest part of building these systems.

- **The Paradox of Expertise:** The better an expert is, the harder it is for them to explain how they do it. Much of human expertise is subconscious intuition.
- **Time & Cost:** Interviewing experts and writing thousands of rules manually is incredibly slow and expensive.
- **Maintenance:** As domains change (e.g., new medical treatments), updating a 10,000-rule system without breaking old rules is a nightmare.

Brittleness and The "Rule Explosion"

Brittleness:

Expert systems lack "Common Sense." If an expert system encounters a situation it lacks a specific rule for, it either crashes or gives a wildly incorrect answer. It cannot "guess" or generalize.

The Rule Explosion:

Writing rules to define basic properties wastes time.

IF animal is a Penguin THEN animal is a Bird.

IF animal is a Bird THEN animal has Feathers.

We needed a better way to represent structural knowledge.

Part 4: Ontological Engineering

Teaching AI the Structure of the World

What is an Ontology?

In philosophy, ontology is the study of "what exists." In Computer Science, it is a formal representation of a domain.

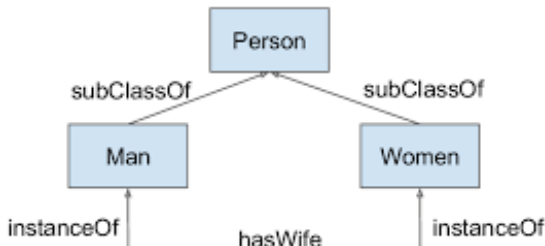
Definition (Tom Gruber, 1993)

An Ontology is an explicit, formal specification of a shared conceptualization.

Instead of writing procedural IF-THEN rules, an ontology maps out the world as a **Graph** of Categories, Objects, and Properties.

The 4 Components of an Ontology

- 1 **Classes (Concepts):** Abstract categories in the domain.
Example: Person, Disease, Hospital
- 2 **Instances (Individuals):** Specific, concrete objects.
Example: Dr. Smith, COVID-19
- 3 **Data Properties (Attributes):** Characteristics of classes.
Example: Age, BloodType
- 4 **Object Properties (Relations):** How instances connect.
Example: Dr. Smith [TREATS] COVID-19



Subsumption (Inheritance)

Ontologies solve the "Rule Explosion" problem through **Subsumption** (Inheritance).

- You define that a Surgeon is a subclass of Doctor.
- You define that a Doctor can PrescribeMedicine.
- The AI's Reasoner automatically infers that a Surgeon can prescribe medicine.

No IF-THEN rules are required. The logic is inherent in the structure of the graph.

Upper Ontologies

- Defines broad, universal, domain-independent concepts.
- Time, Space, Matter, Events, Objects.
- Extremely abstract.
- Examples: BFO (Basic Formal Ontology), SUMO.

Domain Ontologies

- Specific to a particular field.
- Diseases, vehicle parts, university structures.
- Examples: SNOMED CT (Medicine), Gene Ontology (Biology).

Building an Ontology: An OWL Example

How do we actually write an ontology in code? We use languages like **OWL** (Web Ontology Language).

1. Defining Classes & Subclasses

```
Class: Doctor
Class: Patient
Class: Surgeon
    SubClassOf: Doctor
```

2. Defining Object Properties

```
ObjectProperty: treats
    Domain: Doctor
    Range: Patient
```

3. Adding Instances (Data)

```
Individual: DrSmith
    Types: Surgeon
Individual: JohnDoe
    Types: Patient
# Connecting them with a Fact:
Individual: DrSmith
    Facts: treats JohnDoe
```

Activity: Designing a Domain Ontology

Task: Design a micro-ontology for a "University System."

Identify the following:

- ① **3 Classes:** (e.g., Student, Module...)
- ② **1 Subclass Relationship:** (e.g., Undergraduate is a subclass of...)
- ③ **2 Object Properties (Relations):** (e.g., [TEACHES], [ENROLLED_IN])
- ④ **2 Instances (Data):** (e.g., "John Doe", "SE3062")

Discussion: How is drawing this graph more efficient than writing 50 rules?

Activity Solution: University System in OWL

Classes & Properties

Class: Student

Class: Undergraduate

SubClassOf: Student

Class: Module

Class: Lecturer

ObjectProperty: teaches

Domain: Lecturer

Range: Module

ObjectProperty: enrolled_in

Domain: Student

Range: Module

Instances (The Knowledge Graph)

Individual: SE3062

Types: Module

Individual: DrPerera

Types: Lecturer

Facts: teaches SE3062

Individual: Alice

Types: Undergraduate

Facts: enrolled_in SE3062

Notice how the AI now automatically knows that Alice is a Student, even though we only explicitly typed her as an Undergraduate!

The Semantic Web (Web 3.0 Vision)

Created by Sir Tim Berners-Lee, the Semantic Web is an extension of the World Wide Web.

- **Web 1.0/2.0:** Pages are linked for humans to read. Computers don't know what the text means.
- **Semantic Web:** Data is tagged using Ontologies so machines can read, understand, and reason about the content autonomously.

RDF (Resource Description Framework)

The fundamental building block of the Semantic Web.

It stores all knowledge as **Triples**:

Subject $\rightarrow$ Predicate $\rightarrow$ Object

Examples:

- `<SriLanka> <hasCapital> <Colombo>`
- `<Einstein> <bornIn> <1879>`

OWL (Web Ontology Language)

- Built on top of RDF. Adds complex logic.
- Can define constraints: "A person can only have exactly two biological parents."
- Disjointness: "A class cannot be both a Human and a Machine."

SPARQL

- The query language for RDF databases (like SQL for relational databases).
- Allows you to search across massive ontology graphs.

Part 5: Modern Knowledge Graphs

From Academic Ontologies to Big Tech

The Shift to Knowledge Graphs

The term "Ontology" is heavily academic. In the 2010s, industry rebranded these massive, interconnected datasets as **Knowledge Graphs** (KGs).

- A Knowledge Graph is essentially a giant Ontology populated with millions of instances (RDF triples) at scale.
- They form the backbone of modern search engines and recommendation systems.

Real-World KG: Google Knowledge Graph

When you search for "Leonardo da Vinci", Google doesn't just return text matching the string.

- It uses its Knowledge Graph (launched 2012) to return an info panel.
- It *knows* he is an instance of a "Person", his "Date of Birth" is 1452, and his "Sibling" is "Bartolomeo".
- It draws this data by traversing the graph nodes in real-time.

Wikidata & DBpedia

- Open-source knowledge graphs that extract structured data from Wikipedia. Contains billions of RDF triples available for AI to query.

Amazon E-Commerce Graph

- Maps products. "iPhone 15" is a "Smartphone", which has a "Screen Size." Powers exact search filtering (not just keyword matching).

The Cyc Project (1984 - Present)

- The longest-running AI project. An attempt to manually encode all human "common sense" into an ontology.

Part 6: Neuro-Symbolic AI & The Future

Combining Neural Networks with Knowledge Graphs

The Machine Learning Revolution

In the 2010s, Deep Learning took over.

- Instead of writing rules or building ontologies, we fed massive amounts of data into Neural Networks and let them learn the patterns.
- **The Advantage:** Highly scalable, handles messy/unstructured data (images, raw text).

The Problem: LLM Hallucinations

- Large Language Models (like ChatGPT) are amazing at predicting the next word, but they have **no concept of factual truth**.
- They confidently invent facts (Hallucinate) because they are purely statistical, not logical.

Neuro-Symbolic AI: The Best of Both Worlds

We have two paradigms:

- ① **Symbolic AI (Ontologies):** 100% truthful, auditable, logical. But rigid and cannot handle natural language well.
- ② **Neural AI (LLMs):** Fluent, creative, handles natural language perfectly. But prone to hallucination and logical errors.

Neuro-Symbolic AI is the current cutting-edge frontier: Combining the reasoning of Symbolic AI with the fluency of Neural Networks.

Neuro-Symbolic Retrieval-Augmented Generation (Neuro-Symbolic RAG)

The most popular Neuro-Symbolic architecture used in industry today to fix AI hallucinations.

The Neuro-Symbolic RAG Workflow:

- ① User asks the LLM a question: "Who is the CEO of Company X?"
- ② The system intercepts the prompt. It queries a strict, curated **Knowledge Graph / Database** for the exact fact.
- ③ The system retrieves the true fact: [Company X, hasCEO, Jane Doe].
- ④ The fact is injected into the LLM's prompt.
- ⑤ The LLM generates a fluent answer strictly based on the retrieved truth.

- **Expert Systems** separated the Knowledge Base from the Inference Engine, but failed due to the Knowledge Acquisition Bottleneck.
- **Ontologies** structure knowledge hierarchically (Classes, Instances, Properties) to prevent rule explosion and enable shared semantics.
- The **Semantic Web** uses RDF triples and OWL to make data machine-readable.
- Modern **Knowledge Graphs** (Google, Amazon) are massive ontologies that power global search and data integration.
- **Neuro-Symbolic AI & RAG** are the future: combining the strict truth of Knowledge Graphs with the fluency of Large Language Models.

Questions for Thought:

- 1 If you were building a self-driving car, would you use a Rule-Based Expert System, a Machine Learning Neural Network, or a Hybrid? Why?
- 2 Why is an Ontology (Knowledge Graph) more reliable for medical diagnosis than asking ChatGPT?